Supervised Learning (COMP0078) – Lecture 1 Notes
Introduction to Supervised Learning

Shuhan Wang

September 29, 2025

Revision notation: items marked with ⋆ are specifically listed in the revision notes file (`revision25-26-V1.pdf`). All content in this file is examinable.

1 Introduction to the Supervised Learning Problem

1.1 Notations and Background

Definition 1.1 (Supervised Learning Problem⋆)**.** *Given a set of input/output pairs (the **training set**), we wish to compute the functional relationship between input and output:*

$$\mathbf{x} \longrightarrow f \longrightarrow y.$$

- ***Classification**: $y \in \{-1, +1\}$.*
- ***Regression**: $y \in \mathbb{R}$.*
- *$\mathcal{X}$: input space (e.g. $\mathcal{X} \subseteq \mathbb{R}^n$), with elements $\mathbf{x}, \mathbf{x}', \mathbf{x}_i, \ldots$*
- *$\mathcal{Y}$: output space, with elements $y, y', y_i, \ldots$*

Definition 1.2 (Supervised Learning Model⋆)**.** *Given a dataset of (input, label) pairs*

$$\mathcal{S} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)\}, \tag{1}$$

infer a function $f_{\mathcal{S}}$ such that $f_{\mathcal{S}}(\mathbf{x}_i) \approx y_i$ for the training data and, more importantly, for future data $\mathcal{S}' = \{(\mathbf{x}_{m+1}, y_{m+1}), (\mathbf{x}_{m+2}, y_{m+2}), \ldots\}$.

Definition 1.3 (Learning Algorithm)**.** *A **learning algorithm** is a mapping $\mathcal{S} \mapsto f_{\mathcal{S}}$. A new input $\mathbf{x}$ is predicted as $f_{\mathcal{S}}(\mathbf{x})$.*

1.2 Linear Model and Squared Loss

Definition 1.4 (Linear Model)**.** *For any $\mathbf{x} \in \mathcal{X} = \mathbb{R}^n$,*

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = \sum_{i=1}^{n} w_i x_i. \tag{2}$$

The function f is parametrized by $\mathbf{w} \in \mathbb{R}^n$; we also denote $f = f_{\mathbf{w}}$.

***Note:** $f(\mathbf{x}) \in \mathbb{R}$, but we can always recover a classification in $\mathcal{Y} = \{-1, 1\}$ by taking $\operatorname{sign}(f(\mathbf{x}))$. A bias term can be incorporated by augmenting $\mathbf{x} \leftarrow (\mathbf{x}, 1)$.*

Definition 1.5 (Squared Loss⋆)**.** *The error between a prediction p and a target q is measured as*

$$\operatorname{error}(p, q) = (p - q)^2. \tag{3}$$

Definition 1.6 (Least Squares Optimization Problem*)**.** *Using the linear model (Definition 1.4) with the squared loss (Definition 1.5), we seek*

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}\in\mathbb{R}^n} \frac{1}{m}\sum_{i=1}^{m}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2. \tag{4}$$

Remark 1.7 (Convexity of the Squared Loss). The objective in (4) is a convex function of $\mathbf{w}$. For a differentiable convex function, a point $\mathbf{w}^*$ is a minimizer if and only if $\nabla f(\mathbf{w}^*) = \mathbf{0}$. Hence it suffices to find $\hat{\mathbf{w}}$ satisfying

$$\nabla\left(\frac{1}{m}\sum_{i=1}^{m}(\hat{\mathbf{w}}^\top \mathbf{x}_i - y_i)^2\right) = \mathbf{0}.$$

Remark 1.8 (Matrix Notation). Writing the objective in (4) in compact matrix form:

$$\frac{1}{m}\sum_{i=1}^{m}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2 = \frac{1}{m}\|X\mathbf{w} - \mathbf{y}\|^2,$$

where $X \in \mathbb{R}^{m\times n}$ is the design matrix with i-th row $\mathbf{x}_i^\top$, $\mathbf{y} \in \mathbb{R}^m$ is the vector of outputs, and $\|\mathbf{v}\| = \sqrt{\sum_i v_i^2}$ is the Euclidean norm. Using matrix calculus,

$$\nabla_{\mathbf{w}}\|X\mathbf{w} - \mathbf{y}\|^2 = 2X^\top X\mathbf{w} - 2X^\top \mathbf{y}. \tag{5}$$

Theorem 1.9 (Ordinary Least Squares Solution*)**.** *Setting the gradient* (5) *to zero yields the* ***normal equations****:*

$$\hat{\mathbf{w}} = (X^\top X)^{-1}X^\top \mathbf{y}, \tag{6}$$

provided $X^\top X$ is invertible.

Definition 1.10 (Tikhonov Regularization / Ridge Regression)**.** *Introducing a regularization parameter $\lambda > 0$, the* ***ridge regression*** *estimator is*

$$\hat{\mathbf{w}}_\lambda = (X^\top X + m\lambda I)^{-1}X^\top \mathbf{y}, \tag{7}$$

which is the ***unique*** *minimizer of*

$$\frac{1}{m}\sum_{i=1}^{m}(\mathbf{w}^\top \mathbf{x}_i - y_i)^2 + \lambda\|\mathbf{w}\|^2. \tag{8}$$

Note that (7) *generalizes the OLS solution (Theorem 1.9) and is always well-defined since $X^\top X + m\lambda I$ is positive definite for $\lambda > 0$.*

2 Perspectives on Supervised Learning

2.1 Optimal Supervised Learning and the Bayes Estimator*

We assume that data is obtained by sampling i.i.d. from a fixed but unknown probability distribution $P(\mathbf{x}, y)$.

Definition 2.1 (Expected Error / Risk)**.** *The* ***expected error*** *(with respect to the squared loss, Definition 1.5) of a predictor f is*

$$\mathcal{E}(f) := \mathbb{E}\big[(y - f(\mathbf{x}))^2\big] = \int_{\mathcal{X}\times\mathcal{Y}} (y - f(\mathbf{x}))^2\, dP(\mathbf{x}, y). \tag{9}$$

Definition 2.2 (Bayes Estimator*)**.** *The* ***Bayes estimator*** *(or optimal predictor) is*

$$f^* := \arg\min_f \mathcal{E}(f). \tag{10}$$

Problem A: *In order to compute f^* we need to know P, which is unknown.*

Theorem 2.3 (Bayes Estimator for Squared Loss*)**.** *For regression ($\mathcal{Y} = \mathbb{R}$), the Bayes estimator (Definition 2.2) under the expected error (Definition 2.1) is the conditional expectation:*

$$f^*(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}] = \int_{\mathcal{Y}} y \, dP(y \mid \mathbf{x}). \tag{11}$$

Proof of Theorem 2.3. We derive f^* in the discrete case. Define the expected error as

$$\mathcal{E}(f) := \sum_{\mathbf{x}\in\mathcal{X}} \sum_{y\in\mathcal{Y}} (y - f(\mathbf{x}))^2 \, p(\mathbf{x}, y),$$

where $p(\mathbf{x}, y)$ is the probability mass function. Using $p(\mathbf{x}, y) = p(y|\mathbf{x})\, p(\mathbf{x})$,

$$\mathcal{E}(f) = \sum_{\mathbf{x}\in\mathcal{X}} \left[\sum_{y\in\mathcal{Y}} (y - f(\mathbf{x}))^2 \, p(y|\mathbf{x}) \right] p(\mathbf{x}).$$

Define the pointwise expected error $\mathcal{E}(f(\mathbf{x})) := \sum_{y\in\mathcal{Y}} (y - f(\mathbf{x}))^2 \, p(y|\mathbf{x})$, so that

$$\mathcal{E}(f) = \sum_{\mathbf{x}\in\mathcal{X}} \mathcal{E}(f(\mathbf{x}))\, p(\mathbf{x}).$$

To find $f^*(\mathbf{x}')$ for a given $\mathbf{x}'$, differentiate $\mathcal{E}(f)$ with respect to $f(\mathbf{x}')$:

$$\frac{\partial \mathcal{E}(f)}{\partial f(\mathbf{x}')} = \frac{\partial \left[\mathcal{E}(f(\mathbf{x}'))\, p(\mathbf{x}') \right]}{\partial f(\mathbf{x}')},$$

since only the $\mathbf{x} = \mathbf{x}'$ term survives. Setting $z := f(\mathbf{x}')$:

$$\frac{\partial \mathcal{E}(f)}{\partial z} = \frac{\partial}{\partial z} \left[\sum_{y\in\mathcal{Y}} (y - z)^2 \, p(y|\mathbf{x}')\, p(\mathbf{x}') \right] = -2 \sum_{y\in\mathcal{Y}} (y - z)\, p(y|\mathbf{x}')\, p(\mathbf{x}').$$

Setting equal to zero and cancelling $p(\mathbf{x}') > 0$:

$$0 = \sum_{y\in\mathcal{Y}} y \, p(y|\mathbf{x}') - z,$$

which gives $z = \sum_{y\in\mathcal{Y}} y \, p(y|\mathbf{x}')$. Since $\mathbf{x}'$ was generic,

$$f^*(\mathbf{x}) = \sum_{y\in\mathcal{Y}} y \, p(y|\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}].$$

The continuous case follows analogously via the decomposition $P(y, \mathbf{x}) = P(y|\mathbf{x})\, P(\mathbf{x})$:

$$\mathcal{E}(f) = \int_{\mathcal{X}} \left\{ \int_{\mathcal{Y}} (y - f(\mathbf{x}))^2 \, dP(y|\mathbf{x}) \right\} dP(\mathbf{x}),$$

yielding $f^*(\mathbf{x}) = \int_{\mathcal{Y}} y \, dP(y|\mathbf{x})$. □

2.2 Bias–Variance Decomposition

We additionally assume there exists some underlying function F such that

$$y = F(\mathbf{x}) + \epsilon, \tag{12}$$

where ϵ is white noise with $\mathbb{E}[\epsilon] = 0$ and finite variance. Thus the optimal prediction under the squared loss is $f^*(\mathbf{x}) := \mathbb{E}[y \mid \mathbf{x}] = F(\mathbf{x})$ (by Theorem 2.3).

We wish to understand the expected error of an arbitrary learner $A_{\mathcal{S}}(\mathbf{x}')$ (abbreviated $A(\mathbf{x}')$), where the expectation is over both the random training set $\mathcal{S}$ and a fresh sample y' from $P(y|\mathbf{x}')$.

Lemma 2.4 (Variance Identity)**.** *For any random variable Z,*

$$\mathbb{E}[(Z - \mathbb{E}[Z])^2] = \mathbb{E}[Z^2] - \mathbb{E}[Z]^2.$$

Proof. Expanding the left-hand side:

$$\mathbb{E}[(Z - \mathbb{E}[Z])^2] = \mathbb{E}[Z^2 - 2Z\,\mathbb{E}[Z] + \mathbb{E}[Z]^2] = \mathbb{E}[Z^2] - 2\mathbb{E}[Z]^2 + \mathbb{E}[Z]^2 = \mathbb{E}[Z^2] - \mathbb{E}[Z]^2. \quad \square$$

Corollary 2.5 (Second Moment Decomposition)**.** $\mathbb{E}[Z^2] = \mathbb{E}[(Z - \mathbb{E}[Z])^2] + \mathbb{E}[Z]^2$.

Theorem 2.6 (Bias–Variance Decomposition*)**.** *Under the noise model* (12)*, the expected error of learner A at a test point $\mathbf{x}'$ decomposes as:*

$$\mathbb{E}\big[(y' - A(\mathbf{x}'))^2\big] = \underbrace{\mathbb{E}\big[(y' - f^*(\mathbf{x}'))^2\big]}_{\textit{Bayes error (irreducible)}} + \underbrace{\big(f^*(\mathbf{x}') - \mathbb{E}[A(\mathbf{x}')]\big)^2}_{\textit{bias}^2} + \underbrace{\mathbb{E}\big[(A(\mathbf{x}') - \mathbb{E}[A(\mathbf{x}')])^2\big]}_{\textit{variance}}. \tag{13}$$

Proof of Theorem 2.6. Expanding the expected squared error (using independence of y' and $A(\mathbf{x}')$):

$$\mathbb{E}\big[(y' - A(\mathbf{x}'))^2\big] = \mathbb{E}[(y')^2] - 2\,\mathbb{E}[y']\,\mathbb{E}[A(\mathbf{x}')] + \mathbb{E}[A(\mathbf{x}')^2].$$

Applying Corollary 2.5 to both y' and $A(\mathbf{x}')$, and noting $\mathbb{E}[y'] = f^*(\mathbf{x}')$:

$$\begin{aligned}
&= \big\{\mathbb{E}\big[(y' - f^*(\mathbf{x}'))^2\big] + f^*(\mathbf{x}')^2\big\} - 2f^*(\mathbf{x}')\,\mathbb{E}[A(\mathbf{x}')] \\
&\quad + \big\{\mathbb{E}\big[(A(\mathbf{x}') - \mathbb{E}[A(\mathbf{x}')])^2\big] + \mathbb{E}[A(\mathbf{x}')]^2\big\} \\
&= \mathbb{E}\big[(y' - f^*(\mathbf{x}'))^2\big] + \big(f^*(\mathbf{x}') - \mathbb{E}[A(\mathbf{x}')]\big)^2 + \mathbb{E}\big[(A(\mathbf{x}') - \mathbb{E}[A(\mathbf{x}')])^2\big].
\end{aligned}$$

The three terms are respectively the Bayes error (irreducible noise), the squared bias, and the variance. $\square$

Remark 2.7 (Bias–Variance Dilemma). The bias and variance tend to trade off against one another:

- **Many parameters** $\Rightarrow$ greater flexibility to fit data $\Rightarrow$ low bias but high variance.
- **Few parameters** $\Rightarrow$ high bias but the fit varies little across training sets $\Rightarrow$ low variance.

Caveat: This exact decomposition only holds for the squared loss.

2.3 Bayes Estimator for Classification

Definition 2.8 (Bayes Classifier*)**.** *For C-class classification, the **Bayes classifier** is the minimizer of the expected 0–1 loss:*

$$f^*(\mathbf{x}) = \arg\max_{c \in \{1,\dots,C\}} P(Y = c \mid \mathbf{x}). \tag{14}$$

*The **Bayes error rate** is*

$$\int \big(1 - P(Y = f^*(\mathbf{x}) \mid \mathbf{x})\big)\, dP(\mathbf{x}). \tag{15}$$

2.4 No Free Lunch Theorem

Theorem 2.9 (No Free Lunch Theorem*[2, Section 5.1])**.** *Let $A_{\mathcal{S}}$ be any learning algorithm for binary classification $\mathcal{Y} = \{0, 1\}$ over the domain $\mathcal{X}$. Let $m < |\mathcal{X}|/2$. Then there exists some distribution P over $\mathcal{X} \times \{0, 1\}$ such that:*

1. *There exists a function $f : \mathcal{X} \to \{0, 1\}$ with Bayes error 0 (Definition 2.8).*
2. *With probability at least $1/7$ with respect to a random draw $\mathcal{S}$ of m examples from P and a draw of an $(m + 1)$-st example $(\mathbf{x}', y')$,*

$$\text{Prob}[A_{\mathcal{S}}(\mathbf{x}') \neq y'] \geq 1/8.$$

Remark 2.10 (Implication of the NFL Theorem). Theorem 2.9 states that no single learning algorithm can dominate all others across all possible data distributions. However, the hard distributions are constructed adversarially for each algorithm. In practice, we can design algorithms that are effective for the specific problems we care about—this is why supervised learning theory is useful.

3 Hypothesis Space

Since $P(\mathbf{x}, y)$ is unknown, we cannot compute $f^* = \arg\min_f \mathcal{E}(f)$ (Definition 2.2) directly. We are only given a training set $\mathcal{S}$ drawn from P.

Definition 3.1 (Empirical Error)**.** *The **empirical error** (empirical risk) of a predictor f on training set $\mathcal{S}$ (Definition 1.2) is*

$$\mathcal{E}_{\text{emp}}(\mathcal{S}, f) = \frac{1}{m} \sum_{i=1}^{m} (y_i - f(\mathbf{x}_i))^2. \tag{16}$$

Remark 3.2 (Problem B). If we minimize $\mathcal{E}_{\text{emp}}$ over *all* possible functions, we can always find a function with zero empirical error (when the Bayes error is zero). This is problematic because such a function may not generalize to unseen data.

Definition 3.3 (Hypothesis Space*)**.** *A **hypothesis space** $\mathcal{H}$ is a restricted set of candidate functions. We minimize the empirical error (Definition 3.1) within $\mathcal{H}$:*

$$f_{\mathcal{S}} = \arg\min_{f \in \mathcal{H}} \mathcal{E}_{\text{emp}}(\mathcal{S}, f). \tag{17}$$

*This approach is called **Empirical Risk Minimization** (ERM).*

Example 3.4 (Linear Hypothesis Space)**.** The linear hypothesis space is $\mathcal{H} = \{f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} : \mathbf{w} \in \mathbb{R}^n\}$. Applying ERM (Definition 3.3) with the squared loss (Definition 1.5) in this space recovers the least-squares problem (Definition 1.6).

Remark 3.5 (Problem C: Choosing $\mathcal{H}$). Given training data $y_i = f^*(\mathbf{x}_i) + \epsilon_i$, the goal is to approximate f^*. Unless prior knowledge is available (e.g. f^* is linear), we cannot expect $f^* \in \mathcal{H}$. Choosing $\mathcal{H}$ "very large" leads to overfitting.

4 k-Nearest Neighbours

Definition 4.1 (k-Nearest Neighbours Classifier)**.** *Let $N(\mathbf{x}; k)$ be the set of k nearest training inputs to $\mathbf{x}$ and*

$$I_{\mathbf{x}} = \{i : \mathbf{x}_i \in N(\mathbf{x}; k)\}$$

the corresponding index set. For classification:

$$f(\mathbf{x}) = \begin{cases} \textit{class } c_1 & \textit{if } |\{i : y_i = c_1,\ i \in I_{\mathbf{x}}\}| > |\{i : y_i = c_2,\ i \in I_{\mathbf{x}}\}|, \\ \textit{class } c_2 & \textit{otherwise.} \end{cases}$$

For regression: $f(\mathbf{x}) = \frac{1}{k} \sum_{i \in I_{\mathbf{x}}} y_i$ *(a local mean).*

Properties:

- *Closeness is measured using a metric (e.g. Euclidean distance).*
- *Local rule: compute a local majority vote.*
- *Decision boundary is non-linear.*

Remark 4.2 (Probabilistic Interpretation of k-NN). k-NN approximates $P(Y = c \mid \mathbf{x})$ (cf. the Bayes classifier, Definition 2.8) by the empirical frequency $\frac{|\{i : y_i = c,\ i \in I_{\mathbf{x}}\}|}{k}$. The expectation is replaced by averaging over the sample, and conditioning at $\mathbf{x}$ is relaxed to conditioning on a local neighbourhood.

4.1 Asymptotic Optimality of k-NN

Theorem 4.3 (1-NN Near-Asymptotic Optimality*)**.** *As the number of training samples $m \to \infty$, the error rate of the* 1*-nearest neighbour classifier is no more than twice the Bayes error rate (Definition 2.8).*

Proof of Theorem 4.3. Abbreviate $P(c|\mathbf{x}) := P(Y = c|\mathbf{x})$. The Bayes error at $\mathbf{x}$ is $1 - \max_{c \in \{1,\dots,C\}} P(c|\mathbf{x})$.

As $m \to \infty$, the nearest neighbour $\mathbf{x}_{nn} \to \mathbf{x}$, so $P(c|\mathbf{x}_{nn}) \approx P(c|\mathbf{x})$. The expected error rate of 1-NN at $\mathbf{x}$ is

$$\sum_{c=1}^{C} P(c|\mathbf{x})\,[1 - P(c|\mathbf{x})].$$

We need to show:

$$\sum_{c=1}^{C} P(c|\mathbf{x})\,[1 - P(c|\mathbf{x})] \le 2\left[1 - \max_{c \in \{1,\dots,C\}} P(c|\mathbf{x})\right].$$

Let $c^* = \arg\max_c P(c|\mathbf{x})$ and $p^* = P(c^*|\mathbf{x})$. Then

$$\begin{aligned} \sum_{c=1}^{C} P(c|\mathbf{x})\,[1 - P(c|\mathbf{x})] &= \sum_{c \neq c^*} P(c|\mathbf{x})\,[1 - P(c|\mathbf{x})] + p^*(1 - p^*) \\ &\le (C-1) \cdot \frac{1-p^*}{C-1}\left[1 - \frac{1-p^*}{C-1}\right] + p^*(1-p^*) \\ &= (1-p^*)\left[1 - \frac{1-p^*}{C-1} + p^*\right], \end{aligned}$$

where the inequality follows because the sum is maximized when all non-c^* probabilities are equal. Since $p^* \le 1$, we have $1 - \frac{1-p^*}{C-1} + p^* \le 2$, giving the desired bound. □

Theorem 4.4 (k-NN Asymptotic Optimality*)**.** $\mathcal{E}(k(m)\text{-}NN) \to \mathcal{E}(f^*)$ *as $m \to \infty$, provided that:*

1. $k(m) \to \infty$,
2. $\frac{k(m)}{m} \to 0$.

Remark 4.5 (Curse of Dimensionality for k-NN). The rate of convergence in Theorem 4.4 depends exponentially on the input dimension. This is an instance of the **curse of dimensionality** (Definition 4.6).

Definition 4.6 (Curse of Dimensionality*)**.** *The **curse of dimensionality** is the observation that many methods degrade as the dimensionality of data increases. Since volume grows exponentially with dimension, the amount of data required to "cover" the space also increases exponentially.*

Example 4.7 (Volume Ratio)**.** Consider the unit d-dimensional ball centred at the origin versus the $\frac{1}{2}$-radius ball at the origin. The ratio of their volumes is $\left(\frac{1}{2}\right)^d$, which vanishes exponentially as d grows.

5 Model Selection

Example 5.1 (Polynomial Fitting)**.** As an example of hypothesis spaces (Definition 3.3) of increasing complexity, consider regression in one dimension:

$$\begin{aligned}
\mathcal{H}_0 &= \{f(x) = b : b \in \mathbb{R}\},\\
\mathcal{H}_1 &= \{f(x) = ax + b : a, b \in \mathbb{R}\},\\
\mathcal{H}_2 &= \{f(x) = a_1 x + a_2 x^2 + b : a_1, a_2, b \in \mathbb{R}\},\\
&\vdots\\
\mathcal{H}_n &= \left\{ f(x) = \sum_{\ell=1}^{n} a_\ell x^\ell + b : a_1, \ldots, a_n, b \in \mathbb{R} \right\}.
\end{aligned}$$

As the polynomial degree r increases, the empirical error decreases; however, the expected (test) error may increase due to overfitting.

Remark 5.2 (Overfitting vs. Underfitting*). In the polynomial fitting example (Example 5.1), small r leads to **underfitting** (high bias) while large r leads to **overfitting** (high variance). For k-NN (Definition 4.1), the roles are reversed: small k leads to overfitting, while large k leads to underfitting. The training error alone is an unreliable guide—one must also estimate the generalization error.

Definition 5.3 (K-Fold Cross-Validation*)**.** *To select a model (e.g. the polynomial degree r or the number of neighbours k):*

1. *Split the data into K parts of roughly equal size.*
2. *Repeatedly train on $K-1$ parts and test on the part left out.*
3. *Average the errors over the K validation sets to obtain the **cross-validation error**.*

*For $K = m$ this is called **leave-one-out** (LOO) cross-validation. Choose the model (hyperparameter) that minimizes the cross-validation error.*

References

[1] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.

[2] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

[3] T. Cover and P. Hart. Nearest Neighbor Pattern Classification. *IEEE Trans. Information Theory*, 13(1):21–27, 1967.